

■ 2.2.12 Assigning Variables with = and :=

You will have noticed that the input and output lines in *Mathematica* are labeled respectively with `In[n]:=` and `Out[n]=`. We can now explain what these labels mean.

The basic point is that there are two “arrays”, `In` and `Out`. Each input or output line is assigned as the value for an element in these arrays. As a result, you can manipulate `In[n]` and `Out[n]` just like any other subscripted variables. You can for example see all your output lines so far by typing `??Out`. (With most *Mathematica* front ends, you will be able to do this particular thing much more conveniently just by scrolling backwards.)

As their label indicates, output lines are given as immediate assignments for `Out[n]`. Input lines, on the other hand, are given as delayed assignments, using `:=`. This means that the “command” you give on each input line is maintained in an unevaluated form. If you ask for a particular line `In[n]` again, *Mathematica* will automatically re-execute the command that appeared on that line.

Here is an expression.

`In[1]:= (x+1)^2`

`Out[1]= (1 + x)2`

This input line gives the command to expand the latest expression generated.

`In[2]:= Expand[%]`

`Out[2]= 1 + 2 x + x2`

Here is another expression.

`In[3]:= (1 + x + y)^2`

`Out[3]= (1 + x + y)2`

This “re-executes” the command you typed in on line 2. Now `%` refers to the expression `(1 + x + y)^2`.

`In[4]:= In[2]`

`Out[4]= 1 + 2 x + x2 + 2 y + 2 x y + y2`

The difference between `=` and `:=` assignments is just the same for single variables as for functions. If you type `x = value`, then `value` is immediately evaluated, and the result is assigned to the variable `x`. On the other hand, if you type `x := value`, then `value` is not immediately evaluated. Instead, it is evaluated afresh each time `x` is used.

The difference between `=` and `:=` is particularly important when you set up chains of assignments that define values for a sequence of variables.

This assigns a value to *a*.

```
In[5]:= a = 1
Out[5]= 1
```

This defines a *delayed* value for *r*.

```
In[6]:= r := a + 2
```

Every time you ask for *r*, the expression *a* + 2 is evaluated afresh, using whatever the current value of *a* is.

```
In[7]:= r
Out[7]= 3
```

This resets the value of *a*.

```
In[8]:= a = 2
Out[8]= 2
```

The new value of *a* is now used in finding the current value of *r*.

```
In[9]:= r
Out[9]= 4
```

This defines an *immediate* value for *s*. The expression *a* + 3 is immediately evaluated using the current value of *a*. The result is that *s* is simply assigned the value 5.

```
In[10]:= s = a + 3
Out[10]= 5
```

This changes the value of *a* again.

```
In[11]:= a = 7
Out[11]= 7
```

Once again, the value of *r* is found using the new value of *a*.

```
In[12]:= r
Out[12]= 9
```

s, on the other hand, was defined with an *immediate assignment*, so it still has its original value, and is unaffected by the change in the value of *a*.

```
In[13]:= s
Out[13]= 5
```

When you use variables just to give names to intermediate results in your calculations, you will usually want to make immediate assignments, using the = operator.

You can use := to set up variables whose values you want to find in a variety of different “environments”. You can make a delayed assignment for an object *t* using *t* := *value*. Then you can try out various different assignments for the objects on which *t* depends. Every time you ask for *t*, a new value for it is found, based on the current values of the objects on which it depends.

This defines a delayed value for *t*.

```
In[14]:= t := {a, Factor[x^a - d], Factor[x^a + d], d}
```

This sets `a` to 2.

```
In[15]:= a = 2
Out[15]= 2
```

Now `d` is set to 1.

```
In[16]:= d = 1
Out[16]= 1
```

If you ask for `t`, you get the result with the current values of `a` and `d`.

```
In[17]:= t
Out[17]= {2, (-1 + x) (1 + x), 1 + x2, 1}
```

This resets `a`.

```
In[18]:= a = 4
Out[18]= 4
```

This gives the result for `t`, based on the new value of `a`.

```
In[19]:= t
Out[19]= {4, (-1 + x) (1 + x) (1 + x2), 1 + x4, 1}
```

The “variable” `t` in the example above is used much like a “function”. In fact, you could have given all definitions in terms of a function `t[ ]`, with no arguments, instead of in terms of a plain variable `t`. It is probably slightly clearer to use a variable rather than a function with no arguments.

There are some calculations where setting up a sequence of variables with delayed values that depend on other variables can be quite convenient. As a general matter, however, it is usually best to avoid setting things up so that one object has a “hidden”, implicit, dependence on other objects. It is all too easy in cases like this just to forget about some dependence that exists.

It is usually much better to avoid dependence on “global variables”, and instead to specify all dependences explicitly by having them appear as the arguments to functions.